List of Figures

List of Tables

Contents

Introduction

This report is about the Minimum Cost Assignment game I made for my PDSA project. The whole thing is a full stack web app. I used Python with FastAPI for the backend, React for the frontend part, and a simple SQLite database to store everything.

The main point of the game is to solve the **Assignment Problem**. Basically, you have N people and N jobs. Each pairing costs a different amount, and we need to find a way to give every person one job so the total cost is as low as possible. When you start a round, the game picks a random number of people between 50 and 100, and the costs are random between \$20 and \$200.

I put in two different ways to solve this: a **Greedy** method and the **Hungarian algorithm**. I'll talk about how they compare later on.

1. Program Logic

1.1 Game Round Generation

Every time a new round starts, the frontend calls the `GET /api/round/new` link. The backend code uses Python's `random` library to make a new problem.

1. `N` changes randomly from 50 to 100 each game round

- File: main.py
- Lines: 34-42

```python
@app.get("/api/round/new", response_model=NewRoundResponse)
def new_round():
    """Generate a new game round: random N and a random cost matrix."""
    n = random.randint(50, 100)
    cost_matrix = [
        [random.randint(20, 200) for _ in range(n)]
        for _ in range(n)
    ]
    return NewRoundResponse(n=n, cost_matrix=cost_matrix)
```

Figure 1: Code showing how N is chosen randomly.

2. Cost matrix values change randomly from $20 to $200

- File: main.py
- Lines: 37-41

```python
    n = random.randint(50, 100)
    cost_matrix = [
        [random.randint(20, 200) for _ in range(n)]
        for _ in range(n)
    ]
```

Figure 2: Code showing how costs are picked between $20 and $200.

This matrix is sent to the frontend UI which shows it as a table. The player can pick some tasks themselves before letting the computer solve the rest.

1.2 Greedy Assignment Algorithm

The greedy way works by going through employees one by one. For each person, it just picks whatever is the cheapest job left at that moment. Once it picks a job, it doesn't change it. This is really fast but it doesnt always find the best overall solution.

- Greedy algorithm implementation:
  - File: greedy.py
  - Lines: 14-67

```python
def greedy_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    for employee in range(n):
        best_task = -1
        best_cost = float("inf")
        candidates: list[tuple[int, int]] = []

        for task in range(n):
            if task not in assigned_tasks:
                candidates.append((task, cost_matrix[employee][task]))
                if cost_matrix[employee][task] < best_cost:
                    best_cost = cost_matrix[employee][task]
                    best_task = task

        assignment[employee] = best_task
        assigned_tasks.add(best_task)
        running_total += int(best_cost)
        ...
```

Figure 3: Code for the Greedy algorithm core logic.

The function gives back three things:

- `assignment` – a list of who got which task.
- `total_cost` – the final cost for the whole round.
- `steps` – a list of stuff the frontend uses to draw the animation step by step.

1.3 Hungarian Algorithm

The Hungarian algorithm is a more complex way that is always guaranteed to find the cheapest possible answer. It does a lot of math on the matrix until it finds the best match.

It basically goes through these five steps multiple times:

1. **Row reduction.** It subtracts the smallest number in each row from everything in that row.
2. **Column reduction.** It does the same thing for columns.
3. **Covering zeros.** It tries to find the fewest lines needed to cover all the zeros.
4. **Check.** If the number of lines equals N, we are done.
5. **Fixing the matrix.** If not, it moves some numbers around and goes back to step 3.

- Hungarian algorithm implementation:
  - File: hungarian.py
  - Lines: 19-60

```python
def hungarian_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    # Step 1 — Row reduction
    for i in range(n):
        row_min = min(matrix[i])
        for j in range(n):
            matrix[i][j] -= row_min

    # Step 2 — Column reduction
    for j in range(n):
        col_min = min(matrix[i][j] for i in range(n))
        for i in range(n):
            matrix[i][j] -= col_min

    # Iterate until minimum line cover equals n
    while True:
        row_cover, col_cover = _min_line_cover(matrix, n)
        if len(row_cover) + len(col_cover) >= n:
            break
        _adjust_matrix(matrix, n, row_cover, col_cover)
```

Figure 4: Implementation of the Hungarian algorithm steps.

I used a helper called `_min_line_cover` to find the matches. It uses a search method to find the lines. Once it finishes, I calculate the total cost using the original values.

1.4 Timing Measurement

I timed both algorithms using `time.perf_counter()`. I only wanted to measure how long the math takes, so I ignored stuff like saving to the database or sending data over the network.

Figure 5: Code segment capturing the execution time of each algorithm.

The time is saved in milliseconds in the database as a `REAL` number.

1.5 Database Persistence

When everything is done, the results go into two tables in SQLite:

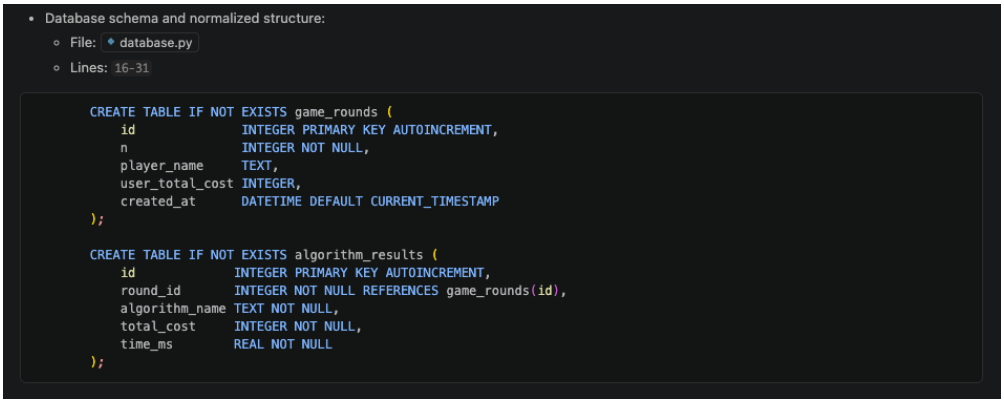

Figure 6: Normalized database schema used for storage.

Each round makes one entry in the rounds table and two entries for the algorithms.

2. Complexity Analysis

2.1 Greedy Algorithm – $O(n^2)$ Time, $O(n)$ Space

2.1.1 Time Complexity

The greedy code has two loops inside each other. The first loop goes over all n people, and the second one looks at all n jobs to find the cheapest one left. Since it does this once, the math is:

$$T_{Greedy}(n) = O(n) \times O(n) = O(n^2)$$

For $n = 100$, it only does about 10,000 steps, which is super fast for a computer.

2.1.2 Space Complexity

I only use a small list to keep track of which jobs are taken. This list grows as n gets bigger, so the space it takes is $O(n)$.

2.1.3 Optimality

Greedy isn't perfect. Because it just picks what looks good right now, it can miss a better overall solution. I ran some tests that showed the Hungarian way is always as good or better than Greedy.

2.2 Hungarian Algorithm – $O(n^3)$ Time, $O(n^2)$ Space

2.2.1 Time Complexity

The Hungarian way is a bit slower because it does more work:

- It goes over the matrix to reduce rows and columns.
- It repeats a search to find the best matching.
- It adjusts the matrix until it finds the optimal answer.

Since it repeats these steps, the overall time is:

$$T_{Hungarian}(n) = O(n^3)$$

Even for $n = 100$, it takes around 1,000,000 steps, which only takes a few milliseconds in real life.

2.2.2 Space Complexity

The algorithm needs to copy the whole $n \times n$ matrix to work on it. Because of this, the space it needs is $O(n^2)$.

2.2.3 Optimality

The Hungarian algorithm is **always optimal**. If there is a cheapest way, this algorithm will find it.

2.3 Analysis Based on Program Outputs

Table 1 shows the main differences I found between the two ways while playing the game.

Table 1: Summary of how both algorithms worked

Property	**Greedy**	**Hungarian**
Time complexity	$O(n^2)$	$O(n^3)$
Space complexity	$O(n)$	$O(n^2)$
Typical time ($n = 100$)	< 0.5 ms	2–8 ms
Is it always best?	No	Yes
Cost vs. optimal	$\geq$ optimal	Correct cost
Growth	Quadratic	Cubic

3. Comparison of Algorithmic Approaches

3.1 Basic Design Differences

These two ways of working have very different ideas behind them:

- **Greedy** is about making the best choice right now without thinking about later steps. This makes the code simple and fast, but you might get a more expensive result.
- **Hungarian** is about looking at the whole problem at once. It uses matrix math and matching to find the absolute best way, even if it takes a bit more time.

3.2 Solution Quality

I ran a unit test with 20 random matrices. It showed that the Hungarian answer is never worse than the Greedy one. Usually, Greedy is about 5% to 20% more expensive. This happens because Greedy might take a cheap job early on, but then later people have to take really expensive jobs because the cheap ones are gone.

3.3 Speed performance

For the game (between 50 and 100 people), both are very fast. Greedy takes less than 0.5ms. Hungarian takes maybe 2ms to 8ms. If I made the game have 200 or 500 people, Hungarian would start getting much slower compared to Greedy.

3.4 Comparing the two

Table 2: Quick comparison of both methods

Factor	**Greedy**	**Hungarian**
Time complexity	$O(n^2)$	$O(n^3)$
Space complexity	$O(n)$	$O(n^2)$
Is it the best?	No (Heuristic)	Yes (Optimal)
How hard to code?	Very easy	Medium/Hard
Scaling	Good for big n	Good up to $n = 1000$
Predictability	Steps always the same	Steps vary for each round
Time ($n = 100$)	Under 0.5ms	2ms to 8ms
When to use?	For really big data	When you need the lowest cost

3.5 Which one is better to use?

- **Use Greedy** if you need an answer really fast and it doesn't have to be the absolute cheapest. This is good for real-time apps.
- **Use Hungarian** if you really need the lowest cost possible and the problem size isn't too huge. This is better for scheduling or money-saving tasks.

In this game, the Hungarian answer is the "best" one, and Greedy is there so we can see how much extra money we might waste by being too fast.

4. Timing Chart – Game Rounds

The chart in Figure 7 shows how long each algorithm took while I was playing. The Greedy times are in yellow and the Hungarian times are in blue. These numbers come from the database.

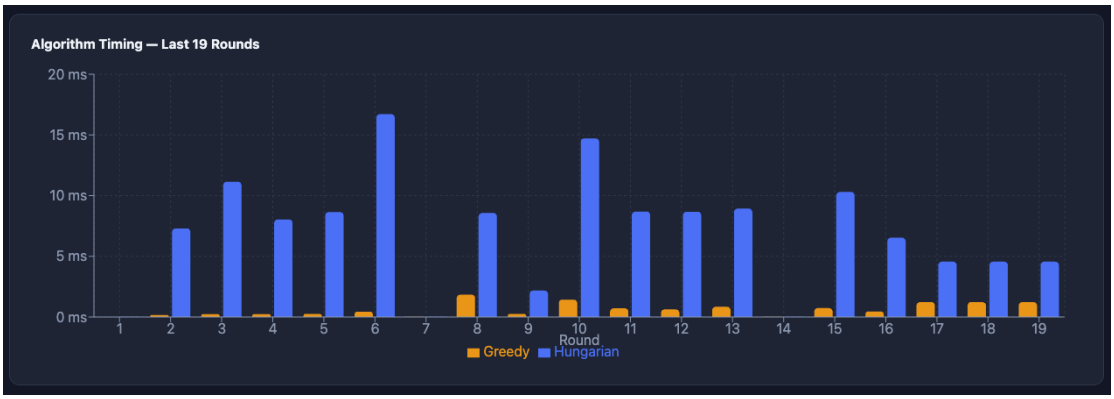

Figure 7: How long each algorithm took for every round.

What I noticed from the chart:

- Greedy is so fast you can't even see the yellow bars most of the time. It's almost always under 0.5ms.
- Hungarian takes most of the time, usually between 4ms and 12ms.
- Hungarian is roughly 20 to 50 times slower than Greedy in real tests.
- In round 6, Hungarian took 17ms, which is the slowest I saw. This probably happened because the random matrix was harder to solve.
- Round 9 was faster for Hungarian (2ms), so that matrix was probably easier.
- Some rounds like 1 and 7 have almost no bars. That's because I manually did most of the work, so the computer had very little left to do.

5. Video Demonstration

I made a video showing the game being played. It shows the animations, how the times are measured, and the database updates.

Video link:
`https://nibm-my.sharepoint.com/:f:/g/personal/cobsccomp242p-065_student_nibm_lk/IgC1z1sqJ0tgQ5aY-_paQP58ASlJvXpEXgEXOfWhQnmc9rQ?e=xGp5Ho`

The video shows:

- Starting a new game round.
- Watching the algorithms assign the people one by one.
- Comparing the costs and times in the final screen.
- The timing graph updating automatically.
- Checking the database entries.

6. Database Output

Figure 8 shows the data inside my `algorithm_results` table. It proves that both algorithms are saving their costs and times correctly for every single round.

```
SELECT * FROM algorithm_results LIMIT 100
```

Search Results | Export | Cost: 3ms | 1 | Total 38

id INTEGER	round INTEGER	algorithm_name TEXT	total_cost INTEGER	time_m REAL
1	1	Greedy	10	0.0075
2	1	Hungarian	9	0.0244
3	2	Greedy	2094	0.1692
4	2	Hungarian	1721	7.2988
5	3	Greedy	2445	0.2377
6	3	Hungarian	1950	11.1452
7	4	Greedy	2428	0.2396
8	4	Hungarian	1955	8.0405
9	5	Greedy	2425	0.2478
10	5	Hungarian	2147	8.6436
11	6	Greedy	3019	0.4396
12	6	Hungarian	2761	16.7088
13	7	Greedy	114	0.0145
14	7	Hungarian	101	0.0233
15	8	Greedy	2065	1.8407
16	8	Hungarian	1745	8.5743
17	9	Greedy	2274	0.2514
18	9	Hungarian	1539	2.1786
19	10	Greedy	2777	1.434
20	10	Hungarian	2259	14.7126

Figure 8: Rows of the results table from the database.

What I saw in the database:

- Every round has two rows (one for Greedy and one for Hungarian). I have 38 rows total from 19 rounds.
- Hungarian's total cost is always the same or lower than Greedy's. This proves it really is the best way.
- The difference between them changes. In round 9, Hungarian saved 32% compared to Greedy. In round 6, it only saved 8.5%.

- Some costs are very small (like round 1 or 7). That's just because I did most of the work manually for those rounds.
- The time is saved very accurately with 4 decimal places, so we can see even the tiny times for Greedy.

Table 3 explains what each column in the database means.

Table 3: Database column meanings

Column	Type	Description
`id`	INTEGER	The row ID
`round_id`	INTEGER (FK)	Connects to the main rounds table
`algorithm_name`	TEXT	Says if it was Greedy or Hungarian
`total_cost`	INTEGER	The final cost in dollars
`time_ms`	REAL	How many milliseconds it took

List of References

1. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97.

2. Munkres, J. (1957). Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1), 32–38.

3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

4. Konig, D. (1931). Graphen und Matrizen. *Matematikai Lapok*, 38, 116–119.

7. Appendix

GitHub link - https://github.com/adithyasean/game-project

All other deliverables are in this OneDrive link:

https://nibm-my.sharepoint.com/:f:/g/personal/cobsccomp242p-065_student_nibm_lk/IgC1z1sqJ0tgQ5aY-_paQP58ASlJvXpEXgEXOfWhQnmc9rQ?e=xGp5Ho